

Detailed Session Summary

Windsurf and Devin — From Intermediate to Advanced

This document is a cleaned, narrative summary of the second day of the AI Sprint and Masterclass conducted by Outskill. The session was hosted by Kartik and led by the mentor Dileep Karri. Day One had set the conceptual foundation; Day Two was about taking those ideas and putting them to work in a real build. The intent of the session was to move learners from an intermediate understanding of agentic coding tools into an advanced, hands on workflow that combines several systems at once: Windsurf as the local AI IDE, Devin as the cloud based autonomous coding agent, and Claude Code as a planning companion sitting beside both of them. The product being built through the live demonstration was nicknamed Superplexity, an attempt at an elevated and more personally curated version of Perplexity.

Session Opening and Welcome

The session opened with Kartik welcoming everyone with warmth and energy. He greeted attendees across time zones with the customary good evening, good morning and good afternoon, and quickly confirmed audio and visual checks before moving into the framing of the day. He reminded the group that they were here for the Windsurf and Devin sprint and that today's instalment was an advancement from Day One, where the focus had been on foundational concepts and a gentle ramp from beginner to intermediate territory.

Kartik underlined a few ground rules that he wanted attendees to internalise before the teaching block began. He asked everyone to keep notebooks and pens ready, to take their notes consistently, and to participate in the chat but to remember that not every question could be answered live. The most relevant questions would be picked up, but the bulk of value would come from active listening and from following the build alongside the mentor on their own machines. He also gave the familiar warning about not trying to go it alone in silence, because a learner who disengages from a live build risks missing the small but important nuances that only become visible when the work is happening in front of them.

With the housekeeping done, Kartik introduced the mentor of the day, describing him affectionately as the favourite mentor of the cohort and an analogies expert. He set the tone by recalling that on Day One

they had discussed recipes; today they would actually be cooking. With that handover line, Dileep Karri came on screen, greeted the group, and began the session proper.

Dileep opened with a small disclaimer that he was on the verge of a headache and that his energy might be a notch lower than usual, but he committed to going as far as he could and asked everyone to bear with him. He thanked the group for the warm welcome and announced that he would begin with a quick recap of Day One before moving into the heart of the new material.

There was an unstated but important pedagogical framing in this opening exchange. The Outskill team had structured the sprint as a deliberate two day arc rather than as a single intensive workshop because the cognitive load of moving from beginner concepts to advanced practice is too high to absorb in one sitting. Day One was a careful warming up of the mental model: vocabulary, distinctions, analogies. Day Two would put the model under load. By stating this clearly at the top of the session, Kartik and Dileep gave learners permission to lean back into the recap rather than feel they had to be already ahead.

Recap of Day One Foundations

The recap was deliberately compact because it was meant to be a launchpad rather than a lesson in itself. Dileep walked the group through the major concepts that had been introduced previously and stitched them together so that learners who had missed Day One would have just enough context to follow the new build.

Vibe Coding Versus Agent Coding

The first distinction recovered from Day One was the difference between vibe coding and agent coding. Vibe coding, in Dileep's framing, is the kind of conversation that happens on platforms such as Bolt, Lovable or Emergent, where the developer is essentially having a dialogue with the AI and the AI is the one building out the requirement from that dialogue. Agent coding, by contrast, is closer to giving the AI a specification or a clear direction; the agents do the building, and the human is the one providing structure, intent and constraints. In vibe coding the AI is interpreting and constructing the requirement; in agent coding the requirement is given more deliberately and the agents execute against it.

GitHub Terminology Revisited

Dileep then revisited the GitHub vocabulary that had been introduced through a pizza restaurant analogy. In that analogy a chef working on a recipe book wanted to share copies of recipes with the sous chef. The challenges that emerge when multiple people work on the same recipe map cleanly onto the problems that version control is designed to solve. Through that story the group had previously learned

what a branch is, what a commit is, what a pull request is and what a merge is. The recap reaffirmed that this terminology would matter throughout Day Two, particularly when Devin started raising its own pull requests against the existing repository.

Specifications Versus Conversations

Another important distinction from Day One was the difference between a specification and a conversation. Dileep had taught this with another food analogy: a specification is like a MasterChef recipe, written out precisely, while a conversation is closer to a grandmother's recipe, full of approximate measurements and shared meaning. The point was that when working with agentic systems, a precise specification yields predictably better outcomes than a loose conversation. Inside the specification one is expected to think about edge cases, about how things might go wrong, about authentication and authorisation, about rate limits, and about error handling.

Authentication, Authorisation, Rate Limits and Error Handling

Dileep reminded the group that authentication and authorisation should be planned, not improvised, even though for the live build of Day Two he was going to set them aside in order to focus on the more interesting parts of the system. He also reminded the group that rate limits are an everyday reality. A developer might be working with Claude, or with a third party API, and at some point will hit a limit. The system needs to know what to do at that point. The same is true for error handling: if error one occurs, if error two occurs, if error three occurs, the application must be told in advance how to respond gracefully rather than simply collapsing.

The Journey of Windsurf

The recap closed with a short history of Windsurf. Windsurf began as a branch of Visual Studio Code, which means that under the surface it shares the same DNA as Cursor, which is also a fork of VS Code. The founding team of Windsurf was then acquired by Google, where they went on to build Antigravity on top of the older Windsurf foundations. The remainder of the Windsurf entity was then acquired by Cognition, the company behind Devin, and from that acquisition emerged what is now spoken of as Windsurf 2.0 along with continued Devin development under the same roof. Dileep mentioned that he expected Devin and Windsurf to keep moving closer together over time, since within Cognition they are essentially being built towards the same end.

Differentiating Windsurf and Devin

Having recovered the recap material, Dileep introduced the central distinction that would shape the rest of the session. Windsurf is an AI IDE; Devin is an autonomous coding agent. That sounds like a small phrasing difference, but in practice it dictates how each tool is used.

Windsurf as an Integrated Development Environment

Windsurf, being an IDE, is the place where the developer actually writes code, sets up the codebase and collaborates with the AI inside the editor. The acronym IDE itself stands for Integrated Development Environment, the kind of tool one opens, sees files in, and edits inside. Windsurf has a side panel agent that the developer can speak with to give direction, ask for changes, and receive feedback while the code is being built. Crucially, Windsurf is local: all the code lives on the developer's own folder on the developer's own machine.

Devin as an Autonomous Coding Agent

Devin, on the other hand, is best thought of as a remote teammate. The way to talk to Devin is by giving it tasks: fix this bug, implement this feature, review this repository for issues. Devin then performs that work autonomously inside its own cloud sandbox. This is the second key difference. Whereas Windsurf runs locally, Devin runs entirely in the cloud, with its own isolated environment in which to clone repositories, run code, write tests and raise pull requests. If a developer shuts down their laptop, Windsurf stops because the IDE on the local machine has gone offline. Devin, however, continues working in the cloud, because the laptop was never the thing doing the work.

The Concept of the Sandbox

Before going further, Dileep paused to make sure that everyone understood what a sandbox is, because this concept underpins the way both Devin and many other agentic tools are designed. He invited anyone who did not know the term to flag it in the chat, and then offered a familiar analogy.

Returning to the restaurant story, he imagined that the sous chef has an idea for a wild new dish and has made changes to the recipe. The chef wants the new recipe tested before it goes anywhere near the main menu. Rather than testing it in the main kitchen, where a mistake could disrupt the regular service, the chef sets up a separate testing kitchen. The sous chef cooks the dish there, the chef tastes it, and only if it is good does it migrate to the main kitchen. A sandbox is exactly that idea applied to code. It is a protected environment in which experiments can be run without affecting anything outside it. For any meaningfully ambitious experiment, the sandbox protects the wider system.

Dileep emphasised that Devin's cloud sandbox is one of the reasons it can be left alone to do significant work. It will not break the developer's local environment, and it will not change the production codebase except through controlled mechanisms such as raising a pull request. That insulation is what gives a developer the confidence to delegate.

Combining Tools — IDE Plus Autonomous Agent

Dileep made an important observation about how tools combine in this space. There is no single right pairing. Windsurf plus Devin is one combination, where Windsurf is the IDE and Devin is the autonomous agent. Cursor plus Claude Code is another combination, in which Cursor plays the role of IDE and Claude Code plays the role of autonomous agent. The shape is the same; only the names of the components change.

This means that learners are not locked into a single stack. Anything can be combined with anything else. Claude Code can in fact be installed inside Windsurf, because Windsurf is a fork of VS Code and VS Code has an official Claude Code extension that works inside any of its forks. JetBrains based IDEs fall into the same bucket, and even with JetBrains it is possible to plug in Claude Code to get an agentic coding experience, or to plug in Devin and use it from there. The shared lineage of these tools means that there is generally a seventy to eighty percent feature overlap between any two of them, and the practical advice is to pick one stack, go deep into it, and only then start exploring lateral combinations.

For this particular session, Dileep chose Windsurf as the IDE and Devin as the autonomous agent, with Claude Code sitting beside both of them to do the heavy planning work. He noted that Devin is very powerful and that part of the reason for that power is its purity of focus. Devin is built purely for software. It is not optimised for non coding tasks. Claude Code, by contrast, is somewhat more general and is at least partially optimised for tasks that sit outside pure software engineering. That makes Claude Code a good companion for the planning phase, where the work is closer to product thinking than to compiling code.

Devin and the Story of Its Founder

To explain why Devin is as good as it is, Dileep took a small detour into the background of its founder, Scott Wu, whom he described as a prodigy and a genius. Scott Wu is a former competitive programmer who, by the time he reached university age, had already won three gold medals at the International Olympiad in Informatics. He came third at the Google Code Jam and won a gold medal at the ICPC, which is one of the most prestigious coding competitions in the world. On Codeforces he holds the rank of

Legendary Grandmaster, which is the top of the competitive programming ladder. He is, in other words, a member of a very small group of people considered among the best coders in the world.

Dileep made the point that this background matters. When the person leading a company building an AI software engineering agent is themselves one of the strongest software engineers in the world, the resulting product tends to reflect a deep understanding of what real software work looks like. Scott Wu has an older brother who is also a Harvard graduate with competitive programming achievements, and together they have built the company with that pedigree. The takeaway for the learners was simple: spend time understanding the founders of the AI companies whose tools you use, because their story usually reveals something about the product.

Where Devin Shines — Migrations and Legacy Code

Dileep then crystallised the working rule for when to reach for Devin versus when to reach for Windsurf. Devin is excellent at modifying and migrating existing code. Suppose a developer has a codebase written in COBOL or Fortran or some other legacy language and wants it moved into a modern stack. Devin has demonstrably cracked this problem, and a significant proportion of its enterprise customers are focused on exactly this kind of migration and refactoring work. Anything that involves taking an existing system and improving it, restructuring it, or porting it across languages is well suited to Devin.

Windsurf, by contrast, is better suited to building new applications and to iterating on them as one goes. When the task is starting from a blank canvas and moving forward, Windsurf is the more comfortable home. For the Day Two build, this division of labour was made explicit. Windsurf would be used to build the new Superplexity application, and Devin would be pointed at an older project to demonstrate its strength on existing code.

Building Superplexity — Setting Up the Plan

The hands on portion of the session began with a clear plan of attack. Dileep would use Claude Code on the side for brainstorming and planning. Once the plan was firm, he would hand it over to Windsurf for execution. The reasoning was that Claude Opus is one of the strongest planning models currently available, and so even though the eventual implementation would happen elsewhere, the planning conversation should benefit from the strongest planner. He stressed that the session was going to spend a meaningful amount of time on the plan itself and that no code would be written for some time. The phrase he returned to several times was that everything so far was just the plan.

Why a Plan First

Dileep emphasised that the plan is what allows downstream execution to be uniform and predictable. If the plan is good, the execution can happen anywhere; if the plan is sloppy, no execution environment can rescue it. He drew the parallel to ordinary management practice, where a good plan from a leader becomes executable by anyone with the muscle to do the work. The same principle holds in agentic coding: refine the plan upstream, then trust the agent to execute it downstream.

Bringing in Claude Code Inside Windsurf

To open Claude Code inside Windsurf, Dileep used the VS Code Claude Code extension, which works inside Windsurf because Windsurf is itself a fork of VS Code. He pointed out that anyone curious about how to install Claude Code inside Windsurf or another VS Code derivative can find clear instructions through a simple search. He then began typing the initial brief into Claude Code, asking it to make a plan for building Superplexity, which he described as an elevated version of Perplexity. He asked Claude Code to ask him clarifying questions and to jam with him on building the plan. He instructed it to take edge cases, error handling and a thinking around what would make Perplexity better into account. He intentionally set authentication aside, since for a personal tool it was not the most interesting concern, and he also noted that he was using OpenRouter as the LLM gateway, which would let him bypass many of the rate limit issues that would otherwise show up.

Brainstorming with the Superpowers Skill

To structure the planning conversation, Dileep used a Claude Code skill called Superpowers, specifically its brainstorming subskill. He explained that Superpowers is a community built collection of skills for Claude Code and that anyone interested could install it by giving Claude Code the GitHub link for it. The repository he pointed to is published under the name Obra Superpowers, and he asked learners to search for it under that name. The skill was visible in his Claude Code as soon as he triggered it, and from that point on Claude Code began conducting what felt like a structured interview with him about the system he wanted to build.

The Opening Reframing

Before firing questions, the skill returned an important reframing. It pointed out that an elevated version of Perplexity actually spans a number of independent subsystems: query understanding, web retrieval, source ranking, content extraction, LLM synthesis with citations, follow ups, conversation memory, accounts and billing, and the user interface. It noted that one cannot design all of that in a single specification, and asked the first orienting question.

Primary Motivation

The first question asked what the primary motivation behind Superplexity was. Was it a learning project, a personal power tool, a productisable prototype, or a specific vertical play? Dileep chose the personal power tool option, telling the skill that he wanted to build something for his own use that could search for the latest information, playbooks and related material.

Where Perplexity Falls Short

The next question asked him to articulate where Perplexity itself fell short. Dileep dictated through Wispr Flow that the new tool should not search the broad open web in the shallow way that Perplexity does. Instead, it should look specifically into Substack articles, Beehiiv articles, very good blogs, and high quality tweets. The thesis he wanted to encode was that Perplexity searches the wrong web. Superplexity should bias retrieval toward high signal independent writing and curated tweets rather than toward search engine optimised blog spam. Claude Code echoed that thesis back to him neatly, confirming that it had understood the framing.

Choosing the Right Sources

With the thesis written, the conversation moved into source access. The skill asked whether the user should give a list of Substacks in advance, or whether every query should go to Google to discover them, or whether a hybrid approach was preferable. Dileep chose the hybrid option as the recommended path, on the grounds that a fixed list would be too rigid and a Google only flow would not differ meaningfully enough from Perplexity itself. For Twitter, the skill noted that the problem was different and would need to be tackled separately, but agreed to handle it.

Handling Twitter and the Apify Decision

The Twitter problem turned out to be the hardest of the early decisions. The official X API is expensive, scraping is fragile, and the cost of paying for a high tier of the X API could easily reach two hundred dollars per month, with the further annoyance that capped queries would still drown in noise. Three options were on the table: curated accounts only, where the user supplies a small list of trusted handles; pay for an X API tier; or a hybrid approach where curated accounts are indexed locally and on demand search is done through a scraping service when broader coverage is needed.

Dileep used the moment to check a tangential question with Claude Code: could Apify help with Twitter scraping? Claude Code confirmed that several popular Apify actors exist that can pull user timelines,

search results and similar feeds. Dileep settled on Apify as the scraping service of choice. He acknowledged openly that he had not prepared this build in advance and was deliberately constructing it on the fly. The point of the live demonstration was to show that the planning skill plus Claude Code is powerful enough that even an unprepared build can converge to a working design.

Choosing the Interaction Model and Deployment Mode

With sources clarified, the conversation turned to the interaction model. The skill offered choices ranging from a simple faster Perplexity feel to a richer research workspace where every query opens a session with the synthesised answer plus the raw sources in the sidebar, to a much barer API and command line interface. Dileep picked the simple faster Perplexity option for now, observing that the bigger workspace could be added later. He also said that the application should run as a local app only, because the build was intended for personal use.

Choosing a local deployment had one important consequence that Claude Code raised explicitly. If the user shut down their laptop, scraping would pause, because nothing would be running in the cloud to keep the work going. Dileep was comfortable with this tradeoff for a personal tool. The skill noted that pausing was different from losing work; scraping would simply resume when the application was started again.

The Synthesis Engine — Picking the Right Models

Next came the synthesis engine, the component that actually reads the sources and writes a cited answer. The skill offered three families of options: a cloud frontier model, a cheap tier model, or some kind of tiered routing arrangement. Dileep flagged that he had already configured an OpenRouter API key, and that he wanted to use a strong recent model. The model he settled on for synthesis was a fresh Claude tier reachable through OpenRouter. He also noted in passing the standard practice of keeping API keys in a dotenv file rather than hardcoded into the application, and reassured anyone unfamiliar with environment variables that this is a small thing they can learn separately.

Blending Multiple Source Pools

With the synthesis engine selected, the planning conversation entered the area Dileep called system logic. At any given moment, Superplexity would have at least four pools that could generate results:

indexed newsletters, indexed tweets, a Google site fallback, and potentially the general web. The skill asked how the system should blend these together, and offered three approaches.

Parallel With LLM Sorting

The first approach was the simplest. Every query would fan out to all sources in parallel, and the LLM would be left to sort out which results to trust. This approach is the easiest to implement but offers the least control.

Tiered With Fallback

The second approach was tiered with fallback. The system would always try newsletters first, and if fewer than a certain number of strong matches were returned, it would expand to indexed tweets, and then finally to Google fallback. This effectively gives newsletters veto power over the result set.

Weighted Blend

The third approach was a weighted blend. All sources would run in parallel, but each set of results would be rescored using a source weight. Newsletters would be weighted at one, tweets at zero point seven, Google fallback at zero point four, and a small classifier would tag each query so that the weights could be adapted to its character. The skill's own recommendation was the weighted blend, and Dileep agreed with the recommendation. He noted again that the entire point of having a strong skill plus Claude Code was that the developer did not have to break their head over which features to build. The skill produced sensible questions; the developer answered them; and the answers became the plan.

Indexing Freshness and Budget Discipline

With blending settled, the skill brought up indexing freshness and the Apify budget. Some curated sources need to be ingested on a schedule. Apify charges per scrape, so the user's choices here have real cost implications. The three options were conservative, balanced and deferred, with conservative meaning very cheap and infrequent scraping, balanced meaning a recommended middle ground, and deferred meaning that scraping would happen only at query time. Dileep picked the balanced recommended option.

He explained that if a system does aggressive scraping it can accumulate significant cost very quickly, which is why this question matters. By choosing balanced, the system would do reasonable refreshes without burning money. He invited learners to think carefully about this kind of decision in their own builds rather than accepting every recommendation by default. He admitted that for the purposes of this

teaching build he was accepting most of the defaults, but in his own real work he would normally push back on individual recommendations and tighten the logic in places where his own judgement told him the default was off.

Edge Cases and Default Behaviours

The planning skill then brought up the explicit handling of edge cases and the default behaviours the system should fall back to when things go wrong. Dileep emphasised that this was the very kind of planning he had warned the group about on Day One: thinking through what happens when reality misbehaves rather than waiting for it to fail and patching afterwards.

The first edge case was empty or very thin results. The proposed default was to synthesise from whatever sources came back and to explicitly tell the user that coverage was thin on this topic, rather than to fabricate confidence. The second case was paywalled content; the proposed behaviour was to cite what was accessible and to flag the paywalled portion clearly. The third case was conflicting sources, where the default would be to surface the disagreement explicitly rather than to silently pick a winner. The fourth case was LLM hallucination, addressed through a post check pass. The fifth case was failures from Apify, OpenRouter or any other external dependency, which should produce graceful degradation rather than crashes. The sixth case was stale curated lists, where a passive feedback loop would help quietly retire sources that had stopped being useful.

Dileep accepted all six defaults as stated and then added one of his own.

The Scraping On Demand Decision

The one default that Dileep wanted to modify was the scraping behaviour. He insisted that scraping should happen on demand only. He did not want a background process to keep firing scrapes whenever it felt like it, because that would drain budget invisibly. Scraping should be triggered only when a user explicitly asked for a search, and even then it should be scoped to what the query needed and stop there.

This led to a clarifying question from the skill. When Dileep said scraping on demand, did he mean all fetching, including free fetches such as RSS feeds, or did he only mean paid scraping? Because RSS is free and instant, background polling on it costs nothing, but a strict reading of his rule could disable even that. Dileep clarified that free background fetching such as RSS was fine, but anything that involved Apify or other paid scraping should happen on demand only. He added that the user would also need to provide an Apify API key for this to work, and the system should accommodate that.

He used this exchange to make a teaching point. The recommendation that the skill produces is not a verdict; it is a starting point. The learner should always feel free to push back on a recommendation and impose their own logic when they believe the default is wrong for their use case.

Architectural Approaches and the Thin Orchestrator

Once the source pools, blending logic, freshness rules and edge cases were settled, the skill moved on to architectural approaches. It put three options on the table: a thin orchestrator, a hybrid retrieval layer, and a pure on demand pipeline. The skill recommended the thin orchestrator, which would use SQLite with full text search indexes for the local cache, a Google programmable search style API for fallback, and an Apify Twitter search actor for tweet retrieval, with source preference lookup happening in a small layer above all of these.

Dileep picked the recommended approach. He observed that the system would have a clear shape: a browser frontend, an orchestrator behind it, SQLite for storage, and then external services for Google search, Apify and OpenRouter. He acknowledged that some of the technical detail was abbreviated, and reminded the group that any other platform would generate a slightly different architecture; the point was not to memorise the diagram but to follow the reasoning.

Dileep then introduced a small tweak. He told the skill that the Google search component was sometimes unreliable in his experience, and that he would prefer to use one of the search models in OpenRouter itself, specifically a Sonar model. He chose Sonar deliberately because Sonar is built by Perplexity, and the goal of Superplexity was to build something on top of and better than Perplexity. Using the same underlying search model felt like a thoughtful continuity. The skill accepted his preference and updated the plan to have Sonar replace Google search entirely.

Switching to a Non Technical Lens

At this point the planning output had become quite technical, with database engines, API actors and search models being discussed in shorthand. Dileep used the opportunity to model a technique that he wanted the non technical members of the audience to take away. He typed into the same conversation that he was a completely non technical person, that he did not understand half of what was being said, and asked the skill to simplify the design for him and to tell him whether they were on the right track.

This is a deliberate move that every learner can use. The same skill that is happy to go deep into technical detail is equally happy to translate its design back into plain English when asked. The skill duly reframed the system in everyday terms. It said that Superplexity could be imagined as a search bar that

lives on the user's laptop. What makes it different from Perplexity is where it looks for answers: trusted newsletters, curated Twitter handles, and the wider web only when needed. It also confirmed that the shape of the design matched what Dileep had described.

Dileep walked the group through this exchange and said that whenever a learner feels their AI agent has slipped into overly technical territory, this is the prompt to bring it back. The skill even wrote a memory note for itself saying that responses should remain in plain English. The lesson was: at every step, the AI itself can be the translator.

Importing a Design System

To make sure the eventual user interface had a coherent look rather than the default agent generated styling, Dileep introduced a small but powerful technique. He told the group about a website he had recently come across called styles dot referrer dot design, where one can copy a complete design system as a Markdown file. He copied that design Markdown into the project as a file called design dot md, then moved back into Windsurf and asked the agent to use design dot md as the reference when building the user interface for Superplexity. This is a clean way to give an agentic build a visual identity without having to hand design every screen.

Curating the Source Lists

The next refinement was to give Superplexity a concrete and curated starting set of sources. Dileep told the agent that he intended to use the tool mainly for AI related questions, so the seed sources should reflect that. He asked the agent to identify the top Substack newsletters covering artificial intelligence, the top Beehiiv newsletters covering artificial intelligence, the top Twitter accounts covering artificial intelligence, and the top ten domains worth indexing on the same topic. He explicitly asked the agent to do the search itself rather than have him hand specify the list.

Watching the agent run those searches in the background was, in his words, the value of letting the planning machinery do the legwork. It would only collect the searches required for the chosen topic, no more and no less. When a question came up in the chat about why Outskill itself was not in the resulting list, Dileep replied light heartedly that Outskill was not yet big enough to qualify.

A Note on Dictation and Pace

Throughout the live build Dileep used a dictation tool called Wispr Flow to speak his prompts rather than type them. This is worth noticing because it changed the texture of the session. The pace at which a

developer can converse with an agent matters more than it first appears. Typing slows the conversation down to the speed of the keyboard, and that friction often pushes people toward shorter, less thoughtful prompts. Speaking the prompt aloud lets the developer think in full sentences, qualify constraints in natural language, and bring more nuance into the conversation without paying a typing tax. For a session built around dialogue with a planning skill, that nuance is exactly what makes the eventual design good.

From Specification to Plan

By this point everything substantive had been decided in conversation, and the skill had begun writing the actual specification. The specification was placed inside the docs folder of the project, under a Superplexity design document. Dileep opened the document so that learners could see it. It contained a clean structure: a product summary, scope, architecture, stack, repository layout, components, storage, retrieval, content features, synthesis, the feedback API and the rest of the plan. Everything had been written, but no code had yet been touched.

What a Markdown File Carries

Dileep used the moment to make a side point about Markdown files in agentic projects. Each Markdown file carries its own role. The design dot md file communicates design intent. A skill dot md file describes a skill. A spec dot md file describes a specification. Treating these as named artefacts with definite purposes helps both the developer and the agent stay oriented.

Specification Versus Plan, Made Concrete

Once the specification was written, the next step was to hand it to the writing plans subskill, which would convert it from English into the more technical, step by step plan that an agent could implement directly. The distinction became sharp at this point. A specification is written in proper English; it is the user requirement gathered in one place. A plan is technical and is written for the agent to execute. The specification is for the developer to verify; the plan is for the agent to consume. Dileep let the writing of the plan run, and once it was done, opened it to show the contrast. Where the specification had used careful prose, the plan was structured in phases, broken down into work packages with the kind of granularity that an autonomous agent can pick up and run with.

Handing the Plan to Windsurf for Execution

With the plan in hand, Dileep returned to Windsurf. He pulled the plan file into his Windsurf workspace using the file picker and gave a simple instruction: implement this. He pointed out that there had been

no transfer of the plan from Claude Code to Windsurf in any awkward sense; Claude Code had written the plan into the same docs folder structure, and Windsurf was simply being asked to reference it. The plan was now a shared artefact between the two systems.

Windsurf accepted the instruction and produced a to do list of its own. It announced that it was going to use a sub agent driven development pattern, where multiple sub agents would work in parallel on different parts of the codebase. Dileep approved the approach and let it run. From the developer's perspective, this was no longer vibe coding. Because so much thought had been invested in the specification and the plan, what was happening now was closer to proper development. Lines of code began to appear on screen as the sub agents wrote them.

Why Not Run Superpowers Inside Windsurf

Dileep paused to anticipate an obvious question. If Superpowers is so useful, why not install it directly inside Windsurf and skip the back and forth with Claude Code entirely? The answer, he said, was that he specifically wanted to use Superpowers with Claude Opus, and at the time of his last check, using Claude Opus inside Windsurf required a separate Anthropic API key, which would have added complexity and cost. Rather than fight that arrangement, he was happy to plan in Claude Code, hand the plan to Windsurf, and let Windsurf implement against it.

Why Use Claude Code for Planning

The next supporting point was the strength of Claude Opus as a planner. In Dileep's experience, Claude Opus is one of the best models available for planning work, and even when the implementation happens elsewhere, the design conversation benefits from being held with the best planner one can reach. He acknowledged a separate consideration: Claude can also be costly when used end to end. In a previous engineering cohort, his Claude Code usage had hit usage limits four separate times. He would run out of tokens, wait several hours, resume, and then run out again. That experience had taught him to think carefully about where Claude was best spent. Using it for the planning phase, and then handing execution to Windsurf with its less expensive token budget, was a sensible workload split. Windsurf is more token efficient than many alternatives for the actual writing of code.

Working with Devin on Existing Code

While Windsurf was busy writing the code for Superplexity, Dileep took the group on a parallel tour of Devin so that the time would not be wasted. He pulled up Devin in the browser, signed in, and showed

the standard task entry interface. He noted that Devin also has a command line interface, but for teaching purposes the browser based view was clearer.

For this demonstration he chose an existing project of his own called Model Console, which he had built in the previous sprint with Claude Code. Model Console is, in his words, a small Perplexity like feature where any query the user types is debated by multiple models in parallel, and the final answer is synthesised from the debate. He pointed Devin at the GitHub repository of Model Console and gave it a clear instruction: look through the repository, see if there are any bugs, and tell him whether there is a way to make the code better than it currently is. He asked Devin to discuss with him before making any changes.

Devin's Review of the Model Console Repository

Within a short period of time Devin produced a thorough review report. The report covered bugs, code quality issues, missing features and improvement ideas. Some of the specific findings included a duplicate prompt being sent to the synthesiser, which wasted tokens and could confuse the judging step, and a mismatch between an example environment file and the guard logic that checked for it. In total Devin identified four bugs and seven code quality issues, and offered a curated list of top improvement ideas alongside potential missing features.

Dileep paused to make a point that anchored this part of the session. Devin is, at its core, a software engineering agent, and so the report it produced read like a report from a strong software engineer. The framing reflects how its founder, a former competitive programmer turned company builder, thinks about software work. That is part of why Devin feels especially well aimed at engineers, and why even non engineers can benefit from running it against their code as long as they are willing to read the more technical sections carefully.

He answered an audience question about whether the bugs Devin found could be forwarded to a tool like Sentry. The answer was yes, but it required setting up a Sentry MCP server and connecting it to the workflow. He noted that this is the kind of extension that is straightforward once one is comfortable with the MCP pattern.

Devin in Practice — Cloud Based Autonomy

Dileep gave Devin the green light to go ahead and fix the issues it had identified. Devin began creating a task list that ran to fifteen items, and started executing them one by one. As it worked, Dileep highlighted what was actually happening underneath. The repository had been cloned into Devin's cloud

sandbox, not onto the local machine, and Devin was running its work entirely on its own infrastructure. This is what allowed it to keep going even if the laptop in front of Dileep was shut down.

Pull Requests and Merging

Devin's preferred way of returning work is via a pull request, which means it creates its own branch on the GitHub repository, commits its changes there, and opens a pull request that the human can then review. If everything looks good, the human merges it; if not, the human asks Devin to revise.

Integration With Slack and Other Engineering Workflows

Dileep mentioned, for the benefit of engineering attendees, that Devin also integrates well with Slack. From Slack, one can ask Devin to take on a bug, and Devin will go off, create its own branch, work the issue and raise a pull request against the main code without any further intervention. For organisations that already have Slack as a common communication surface, this turns Devin into a kind of always available junior teammate. For enterprise use the security posture is also stronger; Devin is not simply pushing code out of the corporate network in the way the casual mode might suggest.

Working on Local Code

Devin can also work on local code, including code uploaded as a zip archive, but Dileep recommended putting the code in GitHub and pointing Devin at it directly. The GitHub centric workflow is cleaner and gives the human a clear trail of branches and pull requests to follow.

Running the First Version of Superplexity

Back on the Windsurf side, the build was progressing in the background. Sub agents were writing code, the system was running its own tests, and occasional errors were popping up and being corrected automatically by Windsurf itself. Dileep emphasised that during this phase he was simply allowing the commands to proceed; the developer's job is to supervise rather than to step into every micro decision, especially once a strong plan is in place.

After some time the Superplexity application reached the point where it could be started. Dileep asked Windsurf to start the application, opened the resulting local website, and was greeted by a simple Superplexity interface asking what he wanted to research today. He typed in a query about Claude Code skills. Because the system had been designed to scrape on demand, the first query took a noticeable moment to process.

Troubleshooting the Empty Database

The first response was not a successful answer; it was an honest message saying that coverage was thin and no usable sources had been found, and suggesting that the user add sources to the configsources file. This was an opportunity for Dileep to demonstrate the troubleshooting loop. He copied the error message, returned to Windsurf, pasted it in, and asked the agent to check what was going on. The agent quickly identified the problem: the local newsletter database was empty, because no indexing had run yet. The system was working exactly as designed, but its starting state had no content to draw from. It then began the indexing process to populate the database with the curated newsletter list.

Dileep used this to make a point about edge cases. Ideally, the empty database scenario should have been caught during planning, since it is a foreseeable initial state for any indexed retrieval system. In practice, a few edge cases always slip through, and the loop of running, hitting the gap, copying the error back, and asking the agent to address it is part of the normal development rhythm. The system did not collapse; it told the user honestly that coverage was thin, which is exactly the behaviour they had chosen during the edge case discussion.

Final MVP and the Importance of a Good Plan

Once the indexing had populated some content, Dileep asked Superplexity another question, this time about the latest capabilities. The system responded with a multi source answer, listing the sources it had used, summarising what Claude Code currently is, what the latest model is, recent feature additions, how it compares to OpenAI's Codex, and broader significance. He noted that the model version mentioned in some of the sources was already slightly out of date, which gave him an opening to remark that this is the nature of working at the edge of fast moving tools.

He acknowledged that the application was still rendering its responses in Markdown rather than in fully styled HTML, and that this was a refinement to address in the next iteration. The point was not that Superplexity was finished, but that an honest MVP had been produced inside a single session. He used the word minimum viable product deliberately. The application was usable, it had a clear shape, and it could be improved through ongoing feedback to the agent.

Dileep took a moment to underline the most important lesson of the entire build. The MVP was decent because the plan had been good. He had spent serious time refining the plan, and once that plan was solid, anyone, or any tool, could execute it. He drew the parallel to management again, noting that strong leaders build strong plans and then trust others with the muscle to carry them out. The same

principle holds in agentic coding. The agent has the muscle; the human's job is to think clearly about what should be done.

Comparing Devin, Claude Code, and Codex

Before closing, Dileep offered a quick comparison of the three major coding agents he had used: Devin, Claude Code and Codex. His take was that all three were good. Each had a place. For the finesse that is required for serious software engineering, he found Devin to be especially strong, in the same way that one might say a specialist outperforms a generalist on the specialist's home ground. For breadth across both coding and non coding tasks, Claude Code and Codex were better suited. The right choice depends on what one is building and how much non coding work surrounds the coding work.

He also flagged the model question. With Devin, the underlying model behaviour is harder to predict because Devin uses what appears to be a mixture of models rather than a single LLM. He recommended that anyone curious about the internals should check the documentation, since the answer might evolve.

His clearest recommendation was that companies with legacy applications, especially those needing migration from older frameworks such as Angular to a modern framework such as React, should look seriously at Devin. The model has been trained well for this kind of work, and the cloud sandbox model makes it possible to take on long migrations without monopolising a developer's local environment.

Practical Patterns Worth Internalising

Across the whole arc of the session there are several recurring patterns that deserve to be drawn out and stated plainly, because they will matter to anyone building with these tools beyond the workshop.

Plan in the Best Room You Can Reach

Even when the eventual execution will happen in a cheaper or faster environment, the planning conversation should be held with the strongest model and the best skill available. The cost of a few extra tokens in the planning phase is trivial compared with the cost of correcting a poorly designed system later. Dileep made this deliberate: he reached for Claude Code with Claude Opus and the Superpowers skill for the planning, and only handed the work to Windsurf once the plan was firm.

Push Back on Defaults When You Have Reason

The planning skill produced recommendations at every step. Dileep accepted most of them for the purpose of this teaching build, but he also overrode the default on scraping, insisting that it happen on

demand only because the cost implication mattered to him. He also tweaked the architecture by replacing Google search with Sonar from OpenRouter for thematic continuity with Perplexity. These are small interventions, but they are what give the final design its character. The lesson is that the recommendation is a starting point, not a verdict.

Use the Translator Whenever the Conversation Drifts

When the conversation got too technical for comfort, Dileep asked the same skill to switch into plain English and to tell him whether they were on the right track. This is a usable technique for everyone. The agent that is happy to talk in stack details and database engines is equally happy to come back to ordinary language when asked. Non technical builders should keep this prompt close at hand and use it whenever their patience for jargon runs out.

Treat Each Markdown File as a Contract

In the agentic workflow that Dileep demonstrated, `design dot md` describes the visual identity, `spec dot md` or the design document describes the user requirement, and the plan files describe the technical work. Each of these is a small contract between the developer and the agent. Naming them clearly and pointing the agent at them by name is a simple discipline that pays off, because it lets the developer reference the right file at the right moment without having to copy and paste large blocks of context.

Read the Error, Don't Just Forward It

When Superplexity returned the thin coverage error, Dileep did forward the error to Windsurf, but he also paused to interpret it himself. The database was empty because no indexing had run yet. That tiny moment of human interpretation, followed by giving the agent the context, is what made the troubleshooting loop work. A developer who simply pastes errors at agents without thinking about what the error means will spend more time in the loop than necessary.

Pick One Stack and Go Deep

Dileep was clear that the goal of teaching multiple tools was awareness, not mastery. Learners are not expected to know Windsurf, Cursor, Antigravity, Devin, Claude Code and Codex equally well. They are expected to pick one stack that suits their workflow, go deep into it, and treat the others as adjacent options to be aware of. This is good advice in any rapidly moving technical landscape. The cost of trying to keep up with everything is that one keeps up with nothing.

Closing Reflections from Dileep

Devin produced its own review back on Model Console while the Superplexity discussion was wrapping up. Dileep showed the GitHub repository and pointed out that Devin had created a separate branch, opened a pull request, and was working its way through fixes one by one. He also showed the wider tooling that GitHub itself now offers, mentioning Sourcery as an example of code review automation that lives directly inside GitHub. The integration with these tools means that Devin's contributions can be reviewed using the same surfaces that a human teammate's contributions would be reviewed with.

He thanked the group warmly for sticking with him through a session that he felt had not been his best because of his headache. He had wanted to complete the topic and produce a working application from end to end, and despite his lowered energy he had managed to do exactly that. He encouraged the group to remember that they did not need to learn every IDE and every agent on the market. One stack, learned deeply, was more valuable than three stacks learned shallowly. The reason he was teaching multiple tools was to give learners awareness of what existed, not to demand mastery of all of it at once.

Announcements — Alumni Forge and the Catalyst Programme

Before signing off, Dileep mentioned that several attendees had been messaging him about a programme called Catalyst, the team's continuation programme for people who wanted to take their work beyond the standard sprints. The Catalyst programme is currently in its base camp phase, conducting introductory sessions for its third cohort. In one or two lines, Catalyst is a programme in which learners go deeper into the skills they have picked up in previous Outskill programmes, and also work substantially on go to market strategy, including monetisation, positioning and launch. It is meant for those who want to move beyond skills and actually build, ship and sell something.

He noted that the programme had a mix of developers and non developers, and that even developers were learning how to think about launching their own products, what services they could provide to others, and how the programme differed from the team's earlier offerings. He acknowledged the kind words from existing participants and asked anyone interested to book a Calendly slot with the team, with a clear note that booking the slot itself does not cost anything; it is simply an opportunity to talk through whether the programme is a fit. He also told a specific attendee by name that he had seen their direct message and that they should write to the team to sort the matter out.

Kartik then closed the session with the formal announcements. He asked for the customary Boom reactions in the chat, thanked the group for their attention, and let them know that more sprints and masterclasses were already being planned for alumni. He acknowledged that the team could not realistically hold a sprint every week, but committed to doing them often. He also asked attendees to fill in a CSAT poll so the team could use the feedback to make the next sessions better.

He reminded everyone of the first product showcase session for Alumni Forge, scheduled for the upcoming Friday, and stressed that this was not an event to be missed. He briefly explained that Catalyst is an extension to the AI Accelerator and AIGF programmes, framing it as a kind of mini MBA and mini AI MBA hybrid that focuses on how to build a project and actually sell it or deliver it to people in the market. He pointed people to the dropped link and the QR code on screen for those who wanted to block a seat, again repeating that blocking a seat does not cost anything and is simply a way to start a conversation with the team about fee structure and fit.

Wrap Up

The session ended on a warm note. Kartik thanked everyone for joining, promised that the team would be back with Alumni Forge on Friday and with another sprint or masterclass in the following weeks. He encouraged the group to keep learning and not to miss out on building with AI. With that, the session closed.

Looking back across the full arc of the day, the most durable takeaways are not the names of the tools used but the patterns the mentor demonstrated. Plan deeply before writing any code. Let a strong planning model interview you rather than trying to think of everything yourself. Distinguish clearly between a specification, which is for the human, and a plan, which is for the agent. Choose tools by their strengths: Windsurf for building new applications, Devin for migrating and improving existing ones, and a strong planning companion such as Claude Code with Superpowers sitting beside both. Treat agentic systems as collaborators that respond to clear direction, and remember that the best results come from learners who refine their plan first and trust the system to execute against it.

This, in essence, is what it means to move from intermediate to advanced in this space. It is not a matter of knowing more buttons; it is a matter of thinking more carefully before the buttons are ever pressed.